Line-by-Line Code Explanation

`hmdbvljepavlms.ipynb`

V-JEPA 2 vs Qwen2.5-VL

HMDB51 — Human Motion Database

51 Action Classes — 6,754 Videos

Thesis: *Comparative Study of Vision-Language Models: VL-JEPA versus Traditional Token-Based VLMs*

Author: Erize Rai

Roll No: 8025006

June 20, 2026

Contents

1 Overview **2**

2 Cell 1: Package Installation **3**

3 Cell 2: Imports and GPU Setup **4**

4 Cell 3: Constants, Class Definitions, and Frame Loading **7**

5 Cell 4: Download HMDB51 Dataset from Hugging Face **10**

6 Cell 5: Construct Dataset as Local File Paths **12**

7 Cell 6: V-JEPA 2 Feature Extraction (6,754 Videos) **13**

8 Cell 7: Qwen2.5-VL Feature Extraction (6,754 Videos) **18**

9 Cell 8: Load Features and Create Train/Eval Split **22**

10 Cell 9: Classifier Training and Evaluation **24**

11 Cell 10: Final Comparison and Results Summary **27**

12 Key Differences Between HMDB51 and FineGym Notebooks **29**

13 Summary of the Experimental Pipeline **29**

Overview

This document provides a detailed line-by-line explanation of every code cell in the `hmdbvljepavlms.i` Jupyter Notebook. The notebook benchmarks two vision models—**V-JEPA 2** (654M parameters) and **Qwen2.5-VL** (7B parameters)—on the **HMDB51** dataset, a large-scale action recognition benchmark with 51 human action categories and 6,754 video clips.

Experimental setup:

- **Dataset**: HMDB51 — 6,754 videos across 51 action classes
- **Method**: Frozen backbone features + traditional ML classifiers (no fine-tuning)
- **Platform**: Kaggle (GPU T4, 15.6 GB)
- **V-JEPA 2**: Feature extraction took $\sim$32 minutes (3.5 vid/s)
- **Qwen2.5-VL**: Feature extraction took $\sim$56 minutes (2.0 vid/s)

Key difference from FineGym notebook: This notebook processes a much larger dataset (6,754 vs. 172 clips) with significantly more classes (51 vs. 4), and downloads the dataset from Hugging Face Hub as a zip archive rather than from YouTube.

Cell 1: Package Installation

```
!pip install -q transformers accelerate bitsandbytes
    peft datasets huggingface_hub
    opencv-python-headless scikit-learn
```

Explanation:

- `!` — Executes a shell command from within the Jupyter notebook.
- `pip install -q` — Installs Python packages quietly (minimal output).
- **Packages installed:**
  - `transformers` — Hugging Face library for loading V-JEPA 2 and Qwen2.5-VL models and their processors.
  - `accelerate` — Enables efficient model loading with device mapping and mixed-precision support.
  - `bitsandbytes` — 4-bit quantization library for fitting the 7B-parameter Qwen model into GPU memory.
  - `peft` — Parameter-Efficient Fine-Tuning library (dependency; not directly used for training).
  - `datasets` — Hugging Face datasets library (not used directly due to schema validation issues; dataset is loaded manually).
  - `huggingface_hub` — Client for downloading files from Hugging Face repositories.
  - `opencv-python-headless` — OpenCV without GUI; used for video frame extraction.
  - `scikit-learn` — ML classifiers and evaluation metrics.
- **Note:** Unlike the FineGym notebook, `yt-dlp` is *not* needed here because HMDB51 is distributed as a downloadable archive, not as YouTube links.

Cell 2: Imports and GPU Setup

```
import torch, sys, os, time, json, gc, random, tempfile
import numpy as np
import torch.nn as nn
from pathlib import Path
from collections import Counter
from PIL import Image
```

Explanation:

- `torch` — PyTorch deep learning framework for model inference, tensor operations, and GPU management.
- `sys` — System-specific parameters and functions.
- `os` — Operating system interface for file/directory operations.
- `time` — Time measurement for speed/ETA calculations during feature extraction.
- `json` — JSON serialization for saving results.
- `gc` — Garbage collector; explicitly called after unloading large models to reclaim memory.
- `random` — Random number generation for reproducibility.
- `tempfile` — Temporary file creation; used by `load_frames_from_bytes` to write video bytes to a temporary file for OpenCV to read.
- `numpy` (as `np`) — Numerical computing for array operations.
- `torch.nn` (as `nn`) — PyTorch's neural network module.
- `Path` — Object-oriented filesystem paths.
- `Counter` — Counts occurrences of items; used for class distribution analysis.
- `Image` (from PIL) — Creates PIL Image objects from NumPy frame arrays.

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, matthews_corrcoef, cohen_kappa_score,
    classification_report, confusion_matrix,
    top_k_accuracy_score,
)
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.ensemble import (RandomForestClassifier,
    GradientBoostingClassifier, StackingClassifier)
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.decomposition import PCA
import cv2
```

Explanation:

Scikit-learn evaluation metrics:

- `accuracy_score` — Fraction of correct predictions.
- `precision_score` — True positives / (True positives + False positives).
- `recall_score` — True positives / (True positives + False negatives).
- `f1_score` — Harmonic mean of precision and recall: $F_1 = 2 \cdot \frac{P \cdot R}{P+R}$.
- `matthews_corrcoef` — MCC; balanced measure even with unequal class sizes. Range: $[-1, +1]$.
- `cohen_kappa_score` — Agreement measure accounting for chance: $\kappa = \frac{p_o - p_e}{1 - p_e}$.
- `classification_report` — Text report with precision, recall, F1, and support per class.
- `confusion_matrix` — $N \times N$ matrix of true vs. predicted labels.
- `top_k_accuracy_score` — Checks if true label is among the top-k predicted classes (important for 51-class problems).

Additional import vs. FineGym:

- `GradientBoostingClassifier` — Imported here (not in FineGym) but ultimately not used in the classifier dictionary. It builds trees sequentially, where each tree corrects the errors of the previous one.

All other imports (StandardScaler, SVC, RandomForest, MLP, LogReg, KNN, PCA, cv2) serve the same purposes as in the FineGym notebook.

```
device = torch.device('cuda' if
    torch.cuda.is_available() else 'cpu')
if device.type == 'cuda':
    gpu = torch.cuda.get_device_name(0)
    props = torch.cuda.get_device_properties(0)
    mem = getattr(props, 'total_memory', None) or
          getattr(props, 'total_mem', 0)
    print(f'GPU: {gpu} ({mem/1e9:.1f} GB)')
else:
    print('WARNING: No GPU!')

SAVE_DIR = '/kaggle/working'
os.makedirs(SAVE_DIR, exist_ok=True)
print(f'Output: {SAVE_DIR}')
```

Explanation:

- `torch.device('cuda' if ...)` — Selects GPU if available, otherwise CPU.
- GPU detection and reporting: prints "GPU: Tesla T4 (15.6 GB)".
- `SAVE_DIR = '/kaggle/working'` — Kaggle's persistent output directory. Files saved here are preserved between sessions and included in the notebook's output artifact.
- `os.makedirs(SAVE_DIR, exist_ok=True)` — Creates the directory if it doesn't exist.

Cell 3: Constants, Class Definitions, and Frame Loading

```
SEED = 42
NUM_FRAMES = 8
IMG_SIZE = 224
TRAIN_SPLIT = 0.8
```

Explanation:

- `SEED = 42` — Global random seed for reproducibility.
- `NUM_FRAMES = 8` — Number of frames to sample from each video for V-JEPA 2.
- `IMG_SIZE = 224` — Spatial resolution of each frame (224×224 pixels).
- `TRAIN_SPLIT = 0.8` — 80/20 train/evaluation split ratio.

```
HMDB51_CLASSES = [
    'brush_hair','cartwheel','catch','chew','clap',
    'climb','climb_stairs','dive','draw_sword',
    'dribble','drink','eat','fall_floor','fencing',
    'flic_flac','golf','handstand','hit','hug',
    'jump','kick','kick_ball','kiss','laugh','pick',
    'pour','pullup','punch','push','pushup',
    'ride_bike','ride_horse','run','shake_hands',
    'shoot_ball','shoot_bow','shoot_gun','sit',
    'situp','smile','smoke','somersault','stand',
    'swing_baseball','sword','sword_exercise','talk',
    'throw','turn','walk','wave',
]
NUM_CLASSES = len(HMDB51_CLASSES)
print(f'{NUM_CLASSES} action classes')
```

Explanation:

- `HMDB51_CLASSES` — Complete list of 51 action categories in the HMDB51 dataset, ordered alphabetically. Each action represents a distinct human motion captured in short video clips.
- The position of each class in this list serves as its integer label (index 0–50).
- `NUM_CLASSES = 51` — Total number of classification categories, significantly more than FineGym's 4.
- **Dataset context**: HMDB51 (Human Motion DataBase) is a widely-used action recognition benchmark collected from movies, YouTube, and other web sources.

```
def load_frames_from_bytes(video_bytes, n_frames=8,
                           size=224):
    with tempfile.NamedTemporaryFile(suffix='.avi',
                                     delete=False) as f:
        f.write(video_bytes)
        tmp = f.name
    try:
        cap = cv2.VideoCapture(tmp)
        if not cap.isOpened(): return None
        total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
        if total < 1: cap.release(); return None
        idxs = np.linspace(0, total - 1, n_frames,
                           dtype=int)
        frames = []
        for i in idxs:
            cap.set(cv2.CAP_PROP_POS_FRAMES, i)
            ok, fr = cap.read()
            if not ok:
                fr = np.zeros((size, size, 3),
                              dtype=np.uint8)
            fr = cv2.cvtColor(fr, cv2.COLOR_BGR2RGB)
            fr = cv2.resize(fr, (size, size))
            frames.append(Image.fromarray(fr))
        cap.release()
        return frames
    finally:
        os.unlink(tmp)
```

Explanation:

- **Key difference from FineGym:** This function accepts raw *bytes* instead of a file path, because the HMDB51 videos are read from disk as binary data.
- `tempfile.NamedTemporaryFile(suffix='.avi', delete=False)` — Creates a temporary file with `.avi` extension. `delete=False` prevents automatic deletion so OpenCV can read it.
- `f.write(video_bytes)` — Writes the raw video bytes to the temporary file.
- `cv2.VideoCapture(tmp)` — Opens the temporary video file for frame extraction.
- `cap.isOpened()` — Checks if the video file was successfully opened.

- `cv2.CAP_PROP_FRAME_COUNT` — Gets the total number of frames.
- `np.linspace(0, total-1, n_frames, dtype=int)` — Computes `n_frames` evenly-spaced frame indices spanning the entire video.
- For each index: seeks to that frame, reads it, converts BGR→RGB, resizes to 224×224, and wraps as a PIL Image.
- If frame reading fails, a black placeholder (zero-filled array) is used.
- `finally: os.unlink(tmp)` — Always deletes the temporary file, even if an exception occurs. This prevents disk space from filling up over 6,754 videos.

```
torch.manual_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)
```

Explanation:

Seeds all three random number generators (PyTorch, NumPy, Python) with `SEED=42` for reproducible results.

Cell 4: Download HMDB51 Dataset from Hugging Face

```
import os
import zipfile
from huggingface_hub import hf_hub_download
from collections import Counter

download_dir = "/kaggle/working/hmdb51_raw"
extract_dir = "/kaggle/working/hmdb51_dataset"
os.makedirs(download_dir, exist_ok=True)
os.makedirs(extract_dir, exist_ok=True)
```

Explanation:

- `zipfile` — Python's built-in ZIP archive handling module.
- `hf_hub_download` — Hugging Face Hub function that downloads individual files from a repository.
- `download_dir` — Directory for the raw downloaded ZIP archive (∼2.12 GB).
- `extract_dir` — Directory where the ZIP contents will be extracted.
- Both directories are created in the persistent `/kaggle/working/` location.

```
archive_path = hf_hub_download(
    repo_id="jili5044/hmdb51",
    filename="hmdb51.zip",
    repo_type="dataset",
    local_dir=download_dir
)
```

Explanation:

- `hf_hub_download` — Downloads a specific file from a Hugging Face repository.
- `repo_id="jili5044/hmdb51"` — The Hugging Face dataset repository containing HMDB51.
- `filename="hmdb51.zip"` — The specific file to download (the raw ZIP archive). This approach *bypasses* the Hugging Face `datasets` library's schema validation, which can fail on complex video datasets.
- `repo_type="dataset"` — Specifies this is a dataset repository (not a model).
- `local_dir=download_dir` — Saves the file to the download directory.
- **Why not use `datasets.load_dataset()`?** The HMDB51 dataset on Hugging Face had schema validation errors (`DatasetGenerationError`), so this manual

download approach is used as a workaround.

```
with zipfile.ZipFile(archive_path, 'r') as zip_ref:
    zip_ref.extractall(extract_dir)
```

Explanation:

- `zipfile.ZipFile(archive_path, 'r')` — Opens the downloaded ZIP file in read mode.
- `zip_ref.extractall(extract_dir)` — Extracts all contents to the extraction directory. The ZIP contains a folder structure: `hmdb51/{action_class}/{video.avi}`.

```
video_classes = []
for root, dirs, files in os.walk(extract_dir):
    for file in files:
        if file.endswith(('.mp4', '.avi', '.mkv')):
            classname = os.path.basename(root)
            video_classes.append(classname)

counts = Counter(video_classes)
print(f'Classes: {len(counts)}')
for name in sorted(counts.keys()):
    print(f'  {name:<20} Samples: {counts[name]:>4}')
```

Explanation:

- `os.walk(extract_dir)` — Recursively traverses all directories and files under the extraction path.
- For each video file (checking extensions `.mp4`, `.avi`, `.mkv`), the parent directory name is treated as the action class label.
- `os.path.basename(root)` — Extracts just the folder name (e.g., "handstand", "brush_hair").
- `Counter` tallies how many videos belong to each class.
- This verification step confirms all 51 classes were successfully extracted.

Output: 51 structural categories found, with sample counts ranging from 102 (catch, kiss, smile, throw) to 548 (walk).

Cell 5: Construct Dataset as Local File Paths

```
extract_dir = "/kaggle/working/hmdb51_dataset"

ds = []
for root, dirs, files in os.walk(extract_dir):
    for file in files:
        if file.endswith(('.mp4', '.avi', '.mkv')):
            video_path = os.path.join(root, file)
            label_name = os.path.basename(root)
            ds.append({
                'video_path': video_path,
                'label': label_name
            })

print(f"Created 'ds' list with {len(ds)} video paths.")
print(f"Sample: {ds[0]}")
```

Explanation:

- **Purpose:** Creates a flat Python list (`ds`) of dictionaries, each containing a video file path and its class label string.
- This replaces what would normally be a Hugging Face `Dataset` object, working around the `DatasetGenerationError`.
- `os.walk(extract_dir)` — Recursively walks through the extracted dataset directories.
- For each video file:
  - `video_path` — Full absolute path to the video file.
  - `label_name` — The parent folder name, which is the action class (e.g., "handstand").
- Each sample is stored as a dictionary: {`'video_path': '...', 'label': '...'`}.
- **Why this approach?** This manual file listing gives full control over data access without relying on the Hugging Face datasets library's schema validation, which was failing.

Output: "Created a local 'ds' list containing 6754 video paths." Sample: {`'video_path': '/kaggle/working/hmdb51_dataset/hmdb51/handstand/Outdoor-Turnen_handsta 'label': 'handstand'`}.

Cell 6: V-JEPA 2 Feature Extraction (6,754 Videos)

```
from transformers import VJEPA2Model,
                         VJEPA2VideoProcessor

VJEPA_FEAT_PATH = os.path.join(SAVE_DIR,
    'vjepa2_hmdb51_features.pt')
VJEPA_CKPT_PATH = os.path.join(SAVE_DIR,
    'vjepa2_hmdb51_ckpt.pt')
CKPT_EVERY = 500
```

Explanation:

- `VJEPA2Model` — V-JEPA 2 (Video Joint Embedding Predictive Architecture 2) by Meta. A self-supervised vision transformer trained by predicting masked video patches. No text supervision is used during training.
- `VJEPA2VideoProcessor` — Handles video preprocessing (normalization, resizing, tensor conversion).
- `CKPT_EVERY = 500` — Save a checkpoint every 500 videos (vs. 100 in FineGym), appropriate for the larger dataset.

```
if os.path.exists(VJEPA_FEAT_PATH):
    print(f'V-JEPA features COMPLETE')
    vjepa_data = torch.load(VJEPA_FEAT_PATH,
        map_location='cpu', weights_only=True)
else:
    vjepa_proc = VJEPA2VideoProcessor.from_pretrained(
        'facebook/vjepa2-vith-fpc64-256')
    vjepa_model = VJEPA2Model.from_pretrained(
        'facebook/vjepa2-vith-fpc64-256',
        torch_dtype=torch.float16,
        attn_implementation='sdpa',
    ).to(device).eval()
```

Explanation:

- **Skip if done:** Checks for previously-saved features to avoid redundant extraction.
- `from_pretrained('facebook/vjepa2-vith-fpc64-256')` — Loads the pre-trained model from Hugging Face:
  - `vith` — Vision Transformer Huge (654M parameters).

  - `fpc64` — Supports up to 64 frames per clip.
  - `256` — 256-pixel resolution.
- `torch_dtype=torch.float16` — Half-precision inference for 50% memory reduction.
- `attn_implementation='sdpa'` — PyTorch 2.0's Scaled Dot-Product Attention; uses Flash Attention when available for $O(N)$ memory complexity instead of $O(N^2)$.
- `.to(device).eval()` — Moves to GPU and sets to inference mode (no dropout, no gradient tracking).

```
    # Resume from checkpoint if exists
    features, labels, start_idx = [], [], 0
    if os.path.exists(VJEPA_CKPT_PATH):
        ckpt = torch.load(VJEPA_CKPT_PATH, ...)
        features = list(ckpt['features'].unbind(0))
        labels = ckpt['labels'].tolist()
        start_idx = int(ckpt['next_idx'])
        del ckpt; gc.collect()
    else:
        print(f'  Starting fresh (0/{len(ds)})')
```

Explanation:

- **Checkpoint resume:** If a partial checkpoint exists, previously extracted features and labels are loaded, and processing continues from where it left off.
- `ckpt['features'].unbind(0)` — Splits the 2D tensor `[N, 1280]` back into a list of `N` individual 1280-dim vectors.
- `ckpt['labels'].tolist()` — Converts label tensor to a Python list.
- `del ckpt; gc.collect()` — Frees checkpoint memory immediately.

```
    for i in range(start_idx, total):
        sample = ds[i]
        video_path = sample['video_path']
        label_name = sample['label']

        if label_name in HMDB51_CLASSES:
            label_id = HMDB51_CLASSES.index(label_name)
        else:
            label_id = int(label_name) if
                label_name.isdigit() else 0

```

```
        with open(video_path, 'rb') as f:
            vbytes = f.read()

        frames = load_frames_from_bytes(vbytes,
            NUM_FRAMES, IMG_SIZE)
        if frames is None: errors += 1; continue
```

Explanation:

- **Main extraction loop** processes all 6,754 videos sequentially.
- `sample['video_path']` — Gets the absolute file path from the dataset list.
- `sample['label']` — Gets the string class name (e.g., "handstand").
- `HMDB51_CLASSES.index(label_name)` — Converts the string label to its integer index (0–50). For example, "brush_hair" → 0, "wave" → 50.
- **Fallback label mapping:** If the label name isn't in the predefined list (edge case), tries parsing it as an integer; defaults to 0.
- `open(video_path, 'rb')` — Opens the video file in binary read mode.
- `f.read()` — Reads the entire video file into memory as bytes.
- `load_frames_from_bytes` — Extracts 8 uniformly-spaced frames from the video bytes (via the temporary file method described in Cell 3).

```
        inputs = vjepa_proc(frames,
                            return_tensors='pt')
        inputs = {k: v.to(device)
                  for k, v in inputs.items()}

        with torch.no_grad(),
             torch.amp.autocast('cuda'):
            out = vjepa_model(**inputs)
            feat = out.last_hidden_state.mean(dim=1)
                   .cpu().float().squeeze(0)

        features.append(feat)
        labels.append(label_id)
```

Explanation:

- `vjepa_proc(frames, return_tensors='pt')` — Preprocesses the 8 PIL Image frames: normalizes pixel values to the model's expected range, converts to a batch tensor of shape `[1, 8, 3, 224, 224]`.

- `{k: v.to(device)}` — Moves all input tensors (pixel values, attention masks) to GPU.
- `torch.no_grad()` — Disables gradient computation; saves memory and speeds up inference.
- `torch.amp.autocast('cuda')` — Automatic mixed precision: uses FP16 for most operations, FP32 where numerically necessary.
- `vjepa_model(**inputs)` — Forward pass through V-JEPA 2.
- `out.last_hidden_state` — Shape `[1, num_patches, 1280]`. V-JEPA 2 divides each frame into patches and produces a 1280-dimensional embedding for each spatiotemporal patch.
- `.mean(dim=1)` — Global average pooling across all patches, producing a single 1280-dim vector that summarizes the entire video.
- `.cpu().float().squeeze(0)` — Moves to CPU, converts to FP32, removes batch dimension. Result: a 1-D tensor of shape `[1280]`.

```
        # Progress tracking every 100 videos
        if (i + 1) % 100 == 0:
            elapsed = time.time() - t0
            done = i - start_idx + 1
            speed = done / elapsed
            eta = (total - i - 1) / speed / 60
            print(f'  [{i+1}/{total}] {len(features)}'
                  f' extracted | {speed:.1f} vid/s |'
                  f' ETA: {eta:.0f}min')

        # Checkpoint every 500 videos
        if (i + 1) % CKPT_EVERY == 0:
            torch.save({
                'features': torch.stack(features),
                'labels': torch.tensor(labels),
                'next_idx': i + 1,
            }, VJEPA_CKPT_PATH)
```

Explanation:

- **Progress reporting** every 100 videos: shows count, extraction speed (vid/s), and estimated time remaining.
- **Checkpointing** every 500 videos: saves all features extracted so far, labels, and the next index. This is critical for the ∼32 minute extraction because Kaggle kernels can time out.

- `torch.stack(features)` — Combines the list of 1280-dim vectors into a 2D tensor `[N, 1280]`.
- `torch.tensor(labels, dtype=torch.long)` — Creates a 1D integer tensor of labels.

After the loop, final features are saved and the checkpoint file is deleted. The model and processor are explicitly deleted and GPU cache is cleared.

Output: V-JEPA DONE: 6754 videos — 1280-dim — 31.8min — 0 errors. Processing speed: $\sim$3.5 vid/s.

Cell 7: Qwen2.5-VL Feature Extraction (6,754 Videos)

```
from transformers import
    Qwen2_5_VLForConditionalGeneration,
    AutoProcessor, BitsAndBytesConfig

QWEN_FEAT_PATH = os.path.join(SAVE_DIR,
    'qwen25vl_hmdb51_features.pt')
QWEN_CKPT_PATH = os.path.join(SAVE_DIR,
    'qwen25vl_hmdb51_ckpt.pt')
CKPT_EVERY = 500
```

Explanation:

- `Qwen2_5_VLForConditionalGeneration` — Qwen2.5-VL 7B Instruct model by Alibaba. An autoregressive vision-language model that can generate text conditioned on visual inputs (images/videos).
- `AutoProcessor` — Automatically loads the correct tokenizer and image/video processor for Qwen.
- `BitsAndBytesConfig` — Configuration for 4-bit weight quantization.

```
bnb = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type='nf4',
    bnb_4bit_use_double_quant=True,
)
qwen_model = Qwen2_5_VLForConditionalGeneration
    .from_pretrained(
    'Qwen/Qwen2.5-VL-7B-Instruct',
    quantization_config=bnb,
    device_map='auto',
    torch_dtype=torch.float16,
)
qwen_proc = AutoProcessor.from_pretrained(
    'Qwen/Qwen2.5-VL-7B-Instruct')
qwen_model.eval()
qwen_dev = next(qwen_model.parameters()).device
```

Explanation:

- **4-bit quantization configuration:**
  - `load_in_4bit=True` — Quantizes all linear layer weights from FP16 (16 bits) to 4 bits, reducing model size from ∼14 GB to ∼3.5 GB.
  - `bnb_4bit_compute_dtype=torch.float16` — During forward pass, weights are dequantized to FP16 for computation (maintains numerical stability).
  - `bnb_4bit_quant_type='nf4'` — NormalFloat4; a data type optimized for normally-distributed neural network weights (empirically better than uniform INT4).
  - `bnb_4bit_use_double_quant=True` — "Nested quantization": the quantization constants (scale factors) are themselves quantized, saving an additional ∼0.4 bits per parameter.
- `device_map='auto'` — Automatically distributes model layers across available GPUs; overflows to CPU RAM if GPU memory is insufficient.
- `qwen_model.eval()` — Sets to evaluation mode (disables dropout).
- `next(qwen_model.parameters()).device` — Determines which device the model's first parameter resides on (typically `cuda:0`).

```
    frames = load_frames_from_bytes(vbytes, 4, IMG_SIZE)

    messages = [{'role': 'user', 'content': [
        {'type': 'video', 'video': frames},
        {'type': 'text', 'text': 'Describe.'},
    ]}]
    text = qwen_proc.apply_chat_template(messages,
        tokenize=False, add_generation_prompt=True)
    inputs = qwen_proc(text=[text], videos=[frames],
        return_tensors='pt', padding=True)
    inputs = {k: v.to(qwen_dev)
              if hasattr(v, 'to') else v
              for k, v in inputs.items()}
```

Explanation:

- **4 frames** per video (vs. 8 for V-JEPA): Qwen's vision encoder generates many tokens per frame, so fewer frames are used to stay within GPU memory limits.
- `messages` — Chat-format input following Qwen's expected structure:
  - `'type': 'video'` — Provides the video frames as visual context.
  - `'type': 'text', 'text': 'Describe.'` — A minimal text prompt. The actual text content doesn't matter much since we extract hidden states, not generated text.

- `apply_chat_template` — Formats the messages into Qwen's chat format with special tokens:

  ```
  <|im_start|>system\nYou are a helpful assistant.
  <|im_end|>\n<|im_start|>user\n<|video|>
  Describe.<|im_end|>\n<|im_start|>assistant\n
  ```

- `qwen_proc(text=..., videos=...)` — Tokenizes text and processes video frames into pixel value tensors, creating `input_ids`, `attention_mask`, and `pixel_values`.
- `hasattr(v, 'to')` — Some inputs may be non-tensor types (e.g., lists); only tensors are moved to GPU.

```
    with torch.no_grad():
        out = qwen_model(**inputs,
                         output_hidden_states=True)
        feat = out.hidden_states[-1].mean(dim=1)
               .cpu().float().squeeze(0)

    features.append(feat)
    labels.append(label_id)
```

Explanation:

- `torch.no_grad()` — Disables gradient tracking for inference efficiency.
- `output_hidden_states=True` — Instructs the model to return the hidden states from *all* transformer layers, not just the final logits. Without this flag, only the output logits for next-token prediction would be returned.
- `out.hidden_states` — A tuple of tensors, one per transformer layer. Each has shape `[1, seq_len, 3584]` where 3584 is Qwen's hidden dimension.
- `out.hidden_states[-1]` — Selects the *last layer's* hidden states, which contain the most semantically rich representations.
- `.mean(dim=1)` — Mean-pools across the entire sequence (both text and visual tokens), producing a single 3584-dimensional feature vector per video.
- `.cpu().float().squeeze(0)` — Moves to CPU, converts to FP32 for compatibility with scikit-learn, and removes batch dimension.
- **Note:** The 3584-dim features from Qwen are $2.8\times$ larger than V-JEPA's 1280-dim features, reflecting the larger model capacity.

The progress reporting (every 50 videos) and checkpointing (every 500 videos) follow the same pattern as V-JEPA 2.

Output: Qwen DONE: 6754 videos — 3584-dim — 56.3min — 0 errors. Processing speed: $\sim$2.0 vid/s (slower than V-JEPA due to the larger model and text token processing overhead).

Cell 8: Load Features and Create Train/Eval Split

```
vjepa_data = torch.load(VJEPA_FEAT_PATH,
    map_location='cpu', weights_only=True)
qwen_data = torch.load(QWEN_FEAT_PATH,
    map_location='cpu', weights_only=True)

vjepa_feat = vjepa_data['features'].numpy()
qwen_feat = qwen_data['features'].numpy()
vjepa_labels = vjepa_data['labels'].numpy()
qwen_labels = qwen_data['labels'].numpy()

n = min(len(vjepa_labels), len(qwen_labels))
vjepa_feat, qwen_feat = vjepa_feat[:n], qwen_feat[:n]
labels = vjepa_labels[:n]
```

Explanation:

- Both feature files are loaded into CPU memory.
- `.numpy()` converts PyTorch tensors to NumPy arrays, which scikit-learn requires.
- `n = min(...)` — Ensures both models have exactly the same number of samples (6,754 in this case, since both had 0 errors).
- Both feature arrays and the label array are truncated to `n` samples.

Output: V-JEPA features: (6754, 1280) — Qwen features: (6754, 3584) — 51 classes.

```
n_tr = int(n * TRAIN_SPLIT)
perm = np.random.RandomState(SEED).permutation(n)
tr_idx, ev_idx = perm[:n_tr], perm[n_tr:]
y_tr, y_ev = labels[tr_idx], labels[ev_idx]
print(f'Train: {len(tr_idx)} | Eval: {len(ev_idx)}')
```

Explanation:

- `n_tr = int(6754 * 0.8) = 5403` — 5,403 training samples, 1,351 evaluation samples.
- `np.random.RandomState(SEED).permutation(n)` — Creates a deterministic random permutation for reproducible splits. Using a separate `RandomState` object ensures the split is independent of prior random operations.
- `tr_idx`, `ev_idx` — Index arrays for training and evaluation.

- `y_tr`, `y_ev` — Corresponding label arrays.

Output: Train: 5403 — Eval: 1351.

Cell 9: Classifier Training and Evaluation

```
def run_classifiers(X_tr, y_tr, X_ev, y_ev, name):
    scaler = StandardScaler()
    X_tr_s = scaler.fit_transform(X_tr)
    X_ev_s = scaler.transform(X_ev)
    pca = PCA(n_components=0.95, random_state=SEED)
    X_tr_p = pca.fit_transform(X_tr_s)
    X_ev_p = pca.transform(X_ev_s)
    print(f'  PCA: {X_tr_s.shape[1]} -> '
          f'{X_tr_p.shape[1]} dims')
```

Explanation:

- **Feature preprocessing pipeline (same as FineGym):**
- `StandardScaler` — Standardizes each feature dimension to zero mean and unit variance: $z_j = \frac{x_j - \mu_j}{\sigma_j}$. This is critical because:
  - SVM's RBF kernel measures distances in feature space; unscaled features bias the kernel.
  - KNN relies on distance metrics; features with larger ranges dominate.
  - PCA finds directions of maximum variance; unscaled features skew the principal components.
- `fit_transform(X_tr)` — Computes statistics from training data and transforms it.
- `transform(X_ev)` — Applies the *same* transformation to evaluation data (prevents data leakage).
- `PCA(n_components=0.95)` — Finds the minimum number of principal components that explain $\geq 95\%$ of the total variance:
  - V-JEPA: $1280 \rightarrow 286$ dimensions (77.7% reduction).
  - Qwen: $3584 \rightarrow 509$ dimensions (85.8% reduction).
- PCA serves dual purposes: (1) reduces computational cost for classifiers, and (2) removes noisy, low-variance feature dimensions.
- **Note:** The `name` parameter is added for identification (not present in FineGym version).

```
    classifiers = {
        'SVM-RBF': SVC(kernel='rbf', C=10,
            gamma='scale', class_weight='balanced',
            random_state=SEED, probability=True),
        'KNN-7': KNeighborsClassifier(n_neighbors=7,
```

```
            n_jobs=-1),
        'RandomForest': RandomForestClassifier(
            n_estimators=500,
            class_weight='balanced',
            random_state=SEED, n_jobs=-1),
        'MLP': MLPClassifier(
            hidden_layer_sizes=(512, 256),
            max_iter=1000, alpha=1e-4,
            random_state=SEED, early_stopping=True,
            learning_rate='adaptive'),
        'LogReg': LogisticRegression(C=1.0,
            class_weight='balanced', max_iter=1000,
            random_state=SEED,
            multi_class='multinomial'),
    }
```

Explanation:

Same five classifiers as FineGym, with identical hyperparameters:

1. **SVM-RBF**: $C = 10$, RBF kernel with $\gamma = \frac{1}{d \cdot \text{Var}(X)}$. With 51 classes, this uses one-vs-one strategy ($\binom{51}{2} = 1275$ binary classifiers). `probability=True` enables Platt scaling for probability estimates (needed for top-5 accuracy).
2. **KNN-7**: $k = 7$ nearest neighbors. Distance is computed in the PCA-reduced feature space.
3. **Random Forest**: 500 trees, balanced class weights (inversely proportional to frequency). Handles the class imbalance in HMDB51 (e.g., "walk" has 548 samples vs. "throw" has 102).
4. **MLP**: Architecture: input $\rightarrow$ 512 neurons $\rightarrow$ 256 neurons $\rightarrow$ 51 outputs. `early_stopping=True` reserves 10% of training data for validation and stops when validation loss stops improving for 10 consecutive epochs.
5. **Logistic Regression**: Multinomial softmax over 51 classes. Uses L2 regularization with $C = 1.0$.

```
    # Additional metric: Top-5 Accuracy
    if prob is not None:
        try:
            t5 = top_k_accuracy_score(y_ev, prob,
                k=5, labels=range(NUM_CLASSES))
        except: t5 = 'N/A'
```

Explanation:

- **Top-5 accuracy** is computed for HMDB51 (not in FineGym) because with 51 classes, a model may rank the correct class among its top predictions even when the top-1 prediction is wrong.
- `top_k_accuracy_score(y_ev, prob, k=5)` — Checks whether the true label appears among the 5 classes with the highest predicted probabilities.
- `labels=range(NUM_CLASSES)` — Ensures all 51 classes are considered even if some are absent from the evaluation set.
- This metric provides a more forgiving assessment: useful when distinguishing similar actions (e.g., "climb" vs. "climb_stairs") is genuinely ambiguous.

The function returns the best classifier name, a comprehensive metrics dictionary, and a per-class classification report.

V-JEPA 2 Results (HMDB51):

- SVM-RBF: 80.8% accuracy, 96.2% top-5, 81.5% F1
- Best classifier identified by F1 macro score
- PCA reduced features from 1280 to 286 dimensions

Qwen2.5-VL Results (HMDB51):

- PCA reduced features from 3584 to 509 dimensions
- Evaluated with same set of classifiers for fair comparison

Cell 10: Final Comparison and Results Summary

This cell produces the final side-by-side comparison table with the following structure:

```
print('FINAL COMPARISON -- HMDB51 (51 Action Classes)')

def row(name, v1, v2, pct=True):
    if pct:
        print(f'  {name:<25} {v1*100:>17.1f}%
              {v2*100:>17.1f}%')
    else:
        print(f'  {name:<25} {v1:>18.3f}
              {v2:>18.3f}')

row('Accuracy', vj_m['accuracy'], qw_m['accuracy'])
row('Top-5 Accuracy', vj_m.get('top5_accuracy', 0),
    qw_m.get('top5_accuracy', 0))
row('Precision (macro)', ...)
row('F1 (macro)', ...)
row('MCC', ..., pct=False)
row("Cohen's Kappa", ..., pct=False)
```

Explanation:

- `row()` — Helper function that prints a formatted comparison row with two model values side-by-side.
- `pct=True` — Multiplies values by 100 and adds % suffix.
- `pct=False` — Prints raw decimal values (used for MCC and Cohen's Kappa).
- `vj_m.get('top5_accuracy', 0)` — Safely retrieves top-5 accuracy (may not exist if probability estimation failed).
- All metrics are printed in a tabular format for easy comparison between V-JEPA 2 and Qwen2.5-VL.

```
winner = 'V-JEPA 2' if vj_f1 > qw_f1
         else 'Qwen2.5-VL'
margin = abs(vj_f1 - qw_f1) * 100
print(f'WINNER: {winner} (by {margin:.1f}% F1)')
```

Explanation:

- Compares the macro F1-scores of both models.

- Declares the model with higher F1 as the winner.
- Reports the margin of victory as a percentage point difference.

```
results = {
    'dataset': 'HMDB51',
    'task': '51-class action recognition',
    'classes': HMDB51_CLASSES,
    'num_videos': n,
    'train': len(tr_idx),
    'eval': len(ev_idx),
    'vjepa2': {'params': '654M',
        'best_clf': vj_name, ...},
    'qwen25vl': {'params': '7B',
        'best_clf': qw_name, ...},
    'winner': winner,
}
results_path = os.path.join(SAVE_DIR,
    'hmdb51_comparison.json')
json.dump(results, open(results_path, 'w'), indent=2)
```

Explanation:

- All experimental results are serialized into a structured JSON file containing:
  - Dataset metadata (name, task, class list, sample counts).
  - Per-model results (best classifier, all metric values).
  - The overall winner.
- `json.dump(..., indent=2)` — Pretty-prints the JSON with 2-space indentation for readability.
- This file serves as a machine-readable record of the experiment for further analysis or inclusion in the thesis.

Key Differences Between HMDB51 and FineGym Notebooks

Aspect	**FineGym**	**HMDB51**
Dataset source	YouTube (via yt-dlp)	Hugging Face Hub (ZIP)
Number of classes	4	51
Number of videos	172 (after download failures)	6,754
Video loading	`load_frames_from_file()`	`load_frames_from_bytes()`
Checkpoint frequency	Every 100 clips	Every 500 videos
Top-5 accuracy	Not computed	Computed
GradientBoosting	Not imported	Imported (unused)
Extraction time (V-JEPA)	$\sim$minutes	$\sim$32 minutes
Extraction time (Qwen)	$\sim$minutes	$\sim$56 minutes
Train/Eval split	137/35	5403/1351
PCA dims (V-JEPA)	$1280 \rightarrow 70$	$1280 \rightarrow 286$
PCA dims (Qwen)	$3584 \rightarrow 97$	$3584 \rightarrow 509$

Table 1: Comparison of notebook design differences.

Summary of the Experimental Pipeline

Both notebooks follow the same five-step pipeline:

1. **Environment Setup:** Install dependencies, configure GPU, set hyperparameters.
2. **Data Acquisition:** Download and prepare the video dataset (YouTube clips for FineGym, ZIP archive for HMDB51).
3. **V-JEPA 2 Feature Extraction:** Load the frozen V-JEPA 2 ViT-H model, extract mean-pooled 1280-dim features from each video, with checkpointing for crash recovery.
4. **Qwen2.5-VL Feature Extraction:** Load the 4-bit quantized Qwen2.5-VL 7B model, construct chat-format inputs with video frames and a minimal text prompt, extract mean-pooled 3584-dim hidden states from the last transformer layer.
5. **Classification and Comparison:** Standardize features, apply PCA, train five traditional ML classifiers (SVM, KNN, Random Forest, MLP, Logistic Regression), evaluate with comprehensive metrics (accuracy, precision, recall, F1, MCC, Cohen's Kappa, confusion matrices), and declare the winner.